

ocean Ai

// session 05 beginner Python

Worlds with depth

Turn planet names into structured data the game can use.

dictionaries

keys

values

// chart the galaxy



Give every planet a **story**.

By the end, your game will read five useful facts about each destination.

01

Model

Store related facts together.

02

Read

Look up a value by its key.

03

Build

Create a galaxy of detailed worlds.

OUTCOME

Your descriptions and encounters drive the journey.



```
// retrieve lists + loops
```

What does this loop visit first?

```
PLANETS = ["Verdantis", "Kragnor"]
```

→

```
PLANETS[0]
```

"Verdantis"

Lists keep order, and index zero means the first item.

A name cannot describe a whole world.

The loop can visit **Verdantis**—but the game still needs danger, water, description, and encounter data.

"Verdantis"

name ✓

description ?

danger ?

water ?

encounter ?

A dictionary stores related facts as **key-value pairs**.

The key names the fact. The value stores the information.

```
planet = {  
    "name": "Verdantis",  
    "description": "Bioluminescent forests cover  
the surface",  
    "danger_level": 1,  
}
```

Read the colon as “maps to.”

"danger_level"

key

:

1

value

SAY IT The key **danger_level** maps to the value 1.

Use the key inside square brackets.

Python finds the matching pair and gives the value back.

```
planet → ["name"] → "Verdantis"
```

```
print(planet["name"])  
print(planet["danger_level"])
```


One dictionary can hold different value types.

The key is always text in our planets. The value depends on what fact we need.

```
planet = {  
    "name": "Verdantis",      # string  
    "danger_level": 1,        # integer  
    "has_water": True,        # boolean  
}
```

string → integer → boolean

STEP 01

Use the same lookup on the left.

Assignment replaces the old value at that key.

```
planet["danger_level"] = 5  
print(planet["danger_level"])
```

Look for **5**. The other facts stay unchanged.



```
// predict exact keys matter
```

What happens if the dictionary has "danger_level", but we ask for "danger"?

KeyError: 'danger'

Python could not find that exact key. Compare spelling and quotes.

```
// combine a list of dictionaries
```

The galaxy stays a list.

Only each item grows from one string into one dictionary.

```
PLANETS = []  
PLANETS.append(verdantis)  
PLANETS.append(kragnor)  
  
print(PLANETS[0]["name"])  
print(PLANETS[1]["encounter"])
```



```
// trace two lookups
```

What does this produce?

PLANETS[1]

then

["encounter"]

"asteroid_field"

Index 1 selects Kragnor; the key selects its encounter.

```
// loop destination is now a dictionary
```

The loop still visits one item at a time.

Now each destination carries all five facts.

```
for destination in PLANETS:  
    print(destination["name"],  
          destination["danger_level"])
```

// contract five required keys

The engine expects the same shape every time.

KEY	TYPE	JOB
name	string	labels the destination
description	string	sets the scene
danger_level	integer 1-5	controls difficulty
has_water	boolean	allows oxygen refill
encounter	string	chooses what happens

Choose an encounter the engine already knows.

empty

Quiet space. A safe rest.

asteroid_field

Hull damage.

raider

Hull and oxygen cost.

trader

Repair and oxygen.

STEP 02

Build Verdantis.

Check every comma before moving on.

constants.py

```
verdantis = {  
    "name": "Verdantis",  
    "description": "Bioluminescent forests cover the  
surface",  
    "danger_level": 1,  
    "has_water": True,  
    "encounter": "empty",  
}
```


STEP 03

Build a riskier world.

Same keys. Different values. Different game result.

```
kragnor = {  
  "name": "Kragnor",  
  "description": "A barren world inside an asteroid  
belt",  
  "danger_level": 3,  
  "has_water": False,  
  "encounter": "asteroid_field",  
}
```

Replace the old list with structured planets.

Set custom planets to True. Keep the galaxy size equal to the list length.

```
USE_CUSTOM_PLANETS = True
PLANETS = []

PLANETS.append({
    "name": "Verdantis",
    "description": "Bioluminescent forests cover the surface",
    "danger_level": 1,
    "has_water": True,
    "encounter": "empty",
})

PLANETS.append({
    "name": "Kragnor",
    "description": "A barren world inside an asteroid belt",
    "danger_level": 3,
    "has_water": False,
    "encounter": "asteroid_field",
})

GALAXY_SIZE = len(PLANETS)
```

Design your galaxy.

Use only lists, dictionaries, values, and the fixed encounter names.

30 minutes

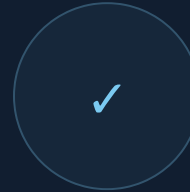
01 Create at least 6 planet dictionaries.

02 Give every planet all five required keys.

03 Mix water, danger levels, and encounter types.

04 Save and run `uv run game.py`.

SUCCESS Your own descriptions and encounters appear during play.



// verify prove the data works

Play through at least three destinations.

DESCRIPTION

Your text appears.

BRANCH

Water and no-water both work.

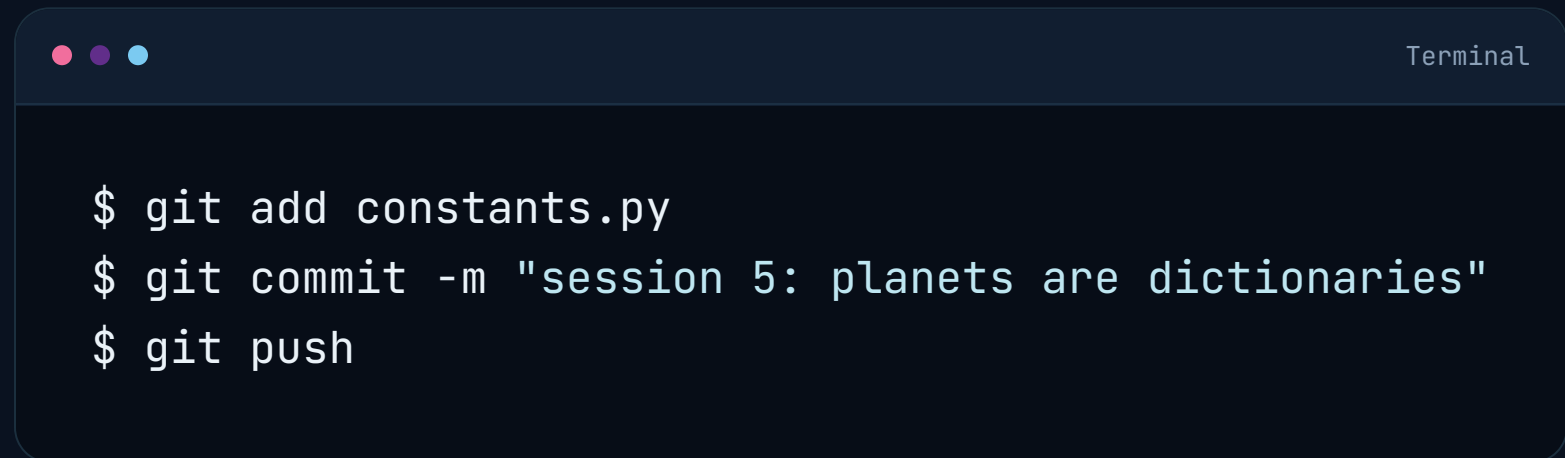
ENCOUNTER

Two different types run.

If you see `KeyError`, copy the missing key exactly.

Save the galaxy's new data model.

Read the commit message before pressing Enter. It should describe what changed.



```
$ git add constants.py
$ git commit -m "session 5: planets are dictionaries"
$ git push
```

CHECK GitHub shows your newest commit.



// reflect explain the upgrade

Why is a dictionary better than a planet name alone?

- 01 What does a key do?
- 02 How does the loop use one destination?
- 03 Which value changed your game most?

ocean Ai

// optional after the core

What else could a planet remember?

Add one unused key such as temperature or population, or build a fixed “death run.”

NEXT: functions give each job a name